

Distributed Smart Warehouse & Multi-Node Fulfillment Orchestration System

1. Project Overview

The **Distributed Smart Warehouse & Multi-Node Fulfillment Orchestration System** is a Python-based microservices backend designed to manage multiple independent warehouses and fulfill customer orders through a distributed workflow.

Unlike a monolithic application, the system is divided into **five independently runnable processes**:

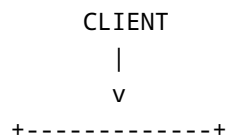
1. API Gateway
2. Inventory Service
3. Order Service
4. Fulfillment Service
5. Notification/Audit Service

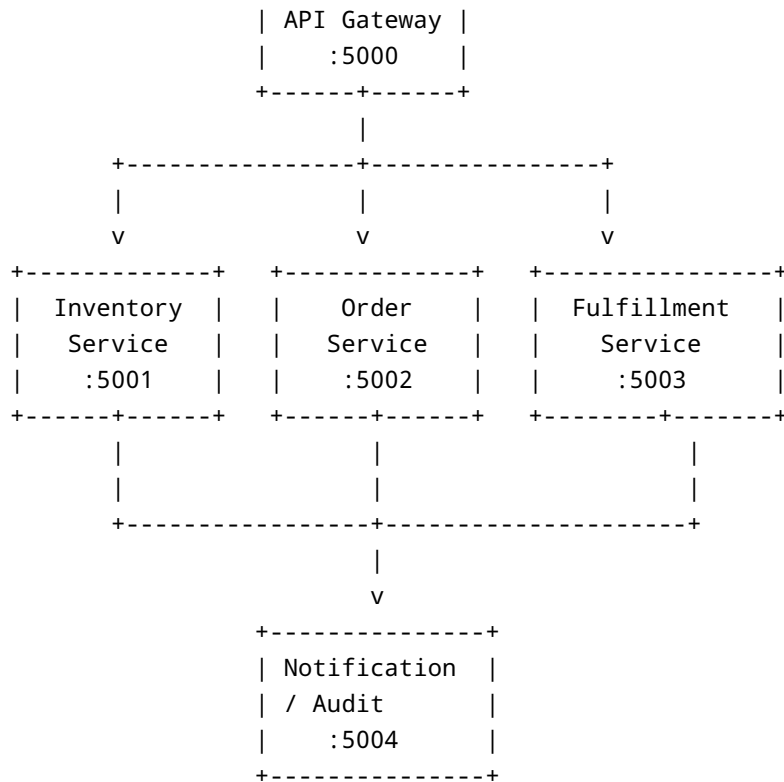
Each backend service owns its own JSON-file database and exposes its own REST API. Services communicate through HTTP rather than sharing memory, databases, or business-logic imports.

The system implements distributed-systems concepts including:

- Microservices architecture
- REST APIs
- Saga pattern
- Compensating transactions
- Idempotency
- Retry with exponential backoff
- Circuit breaker
- Atomic JSON persistence
- Concurrent file protection
- Event-based auditing
- API Gateway aggregation
- Automated testing with pytest

2. System Architecture





Service Responsibilities

Service	Port	Responsibility
API Gateway	5000	Client entry point and response aggregation
Inventory Service	5001	Warehouses and stock
Order Service	5002	Customer orders
Fulfillment Service	5003	Saga orchestration and shipment
Audit Service	5004	Events and audit history

Each service runs as a separate operating-system process.

3. Main Features

Inventory Management

- Create warehouses
- Add inventory items
- Track SKU quantities

- Track reserved quantities
- Query inventory across warehouses
- Reserve stock
- Release reservations
- Commit reservations
- Idempotent stock mutations

Order Management

- Create orders
- Retrieve orders
- List orders
- Filter orders by status
- Update order status
- Cancel orders
- Prevent cancellation after shipment

Fulfillment

- Start fulfillment Saga
- Reserve stock for every order line
- Select an eligible warehouse
- Schedule shipment route
- Track Saga state
- Compensate successful steps when a later step fails
- Retry failed Sagas
- Retry downstream requests
- Circuit breaker protection

Audit

- Receive events from services
- Store events append-only
- Search order history
- Search Saga history
- Generate system activity reports

Gateway

- Single client-facing API
 - Route requests to backend services
 - Aggregate order information
 - Aggregate health information
 - Provide system-wide report
-

4. Technologies

- Python 3.12+
- Flask
- Requests
- Pytest
- JSON files
- Python threading locks
- HTTP REST APIs
- Git/GitHub
- Docker Compose (optional bonus)

No Redis, RabbitMQ, Kafka, PostgreSQL, Kubernetes, Docker Swarm, or managed cloud infrastructure is required.

5. Folder Structure

```
DistributedWarehouseSystem/
|
├── gateway/
│   ├── main.py
│   └── router.py
|
├── inventory_service/
│   ├── main.py
│   ├── manager.py
│   ├── exceptions.py
│   ├── models/
│   │   ├── warehouse.py
│   │   └── inventory_item.py
│   └── data/
│       ├── warehouses.json
│       ├── items.json
│       ├── reservations.json
│       └── idempotency.json
|
├── order_service/
│   ├── main.py
│   ├── manager.py
│   ├── exceptions.py
│   ├── models/
│   │   └── order.py
│   └── data/
```

```
├── orders.json
├── fulfillment_service/
│   ├── main.py
│   ├── manager.py
│   ├── circuit_breaker.py
│   ├── exceptions.py
│   ├── models/
│   │   ├── saga.py
│   │   └── shipment_route.py
│   └── data/
│       ├── sagas.json
│       └── routes.json
├── audit_service/
│   ├── main.py
│   ├── manager.py
│   ├── models/
│   │   └── audit_event.py
│   └── data/
│       └── events.json
├── shared/
│   ├── config.py
│   ├── exceptions.py
│   ├── http_client.py
│   └── store.py
├── tests/
│   ├── test_inventory.py
│   ├── test_order.py
│   ├── test_saga.py
│   └── test_circuit_breaker.py
├── facility_report.txt
├── requirements.txt
├── Dockerfile
├── docker-compose.yml
├── .gitignore
└── README.md
```

6. Installation

Clone the repository:

```
git clone YOUR_GITHUB_REPOSITORY_URL
cd DistributedWarehouseSystem
```

Create a virtual environment:

Windows

```
python -m venv .venv
.venv\Scripts\activate
```

Linux/macOS

```
python3 -m venv .venv
source .venv/bin/activate
```

Install dependencies:

```
pip install -r requirements.txt
```

7. Starting the Services

The five processes use the following ports:

```
Gateway      → 5000
Inventory    → 5001
Order        → 5002
Fulfillment  → 5003
Audit        → 5004
```

Open five terminals.

Terminal 1 — Inventory Service

```
python inventory_service/main.py
```

Terminal 2 — Order Service

```
python order_service/main.py
```

Terminal 3 — Audit Service

```
python audit_service/main.py
```

Terminal 4 — Fulfillment Service

```
python fulfillment_service/main.py
```

Terminal 5 — API Gateway

```
python gateway/main.py
```

The Gateway is then available at:

```
http://localhost:5000
```

8. API Reference

API Gateway

Create Warehouse

```
POST /warehouses
```

Example:

```
curl -X POST http://localhost:5000/warehouses  
-H "Content-Type: application/json"
```

```
-d "{\"id\":\"WH1\",\"name\":\"Islamabad Central\",\"region\":\"Islamabad\",
  \"address\":\"I-9 Islamabad\"}"
```

Example response:

```
{
  "id": "WH1",
  "name": "Islamabad Central",
  "region": "Islamabad",
  "address": "I-9 Islamabad",
  "item_ids": []
}
```

Add Inventory

```
POST /warehouses/{id}/items
```

Example:

```
curl -X POST http://localhost:5000/warehouses/WH1/items
-H "Content-Type: application/json"
-d "{\"id\":\"ITEM1\",\"sku\":\"LAPTOP-01\",\"description\":\"Laptop\",
  \"quantity_on_hand\":20,\"unit_cost\":1200}"
```

Check Inventory

```
GET /inventory/{sku}
```

Example:

```
curl http://localhost:5000/inventory/LAPTOP-01
```

Create Order

```
POST /orders
```

Example:

```
curl -X POST http://localhost:5000/orders
-H "Content-Type: application/json"
-d "{\"customer_name\":\"Saad\",\"customer_address\":\"Islamabad\",\"lines\":
[{\\"sku\":\"LAPTOP-01\",\"quantity\":2}]}"
```

Creating an order starts the fulfillment Saga asynchronously.

Get Full Order Status

```
GET /orders/{id}
```

Example:

```
curl http://localhost:5000/orders/ORDER_ID
```

The Gateway aggregates:

- Order Service response
 - Fulfillment Saga status
 - Audit history
-

Cancel Order

```
POST /orders/{id}/cancel
```

Example:

```
curl -X POST http://localhost:5000/orders/ORDER_ID/cancel
```

An already shipped order cannot be cancelled.

Retry Failed Saga

```
POST /sagas/{id}/retry
```

Example:

```
curl -X POST http://localhost:5000/sagas/SAGA_ID/retry
```

Only failed Sagas can be retried.

Order History

```
GET /orders/{id}/history
```

Example:

```
curl http://localhost:5000/orders/ORDER_ID/history
```

Returns the chronological audit trail.

System Report

```
GET /report
```

Example:

```
curl http://localhost:5000/report
```

The report contains system-level information such as:

- Total warehouses
 - Total orders
 - Orders by status
 - Audit event information
 - Circuit breaker state
 - Timestamp
-

Health Check

```
GET /health
```

Example:

```
curl http://localhost:5000/health
```

The Gateway checks all four backend services and reports their availability.

9. Saga Design

The Fulfillment Service is responsible for orchestration.

The basic workflow is:

```
ORDER CREATED
  |
  v
PENDING
  |
  v
RESERVE STOCK
  |
  v
ASSIGN WAREHOUSE
  |
  v
SCHEDULE ROUTE
  |
  v
FULFILLING
  |
  v
COMPLETED
```

For each order line, the Fulfillment Service checks inventory and selects an eligible warehouse.

The current warehouse selection heuristic is:

1. Warehouse must contain the requested SKU.
2. Available quantity must be sufficient.
3. Select the lowest unit-cost eligible warehouse.
4. Use warehouse ID as the tie-breaker.

10. Compensation Logic

Because the system has no distributed database transaction, a failure can happen after previous operations have succeeded.

Example:

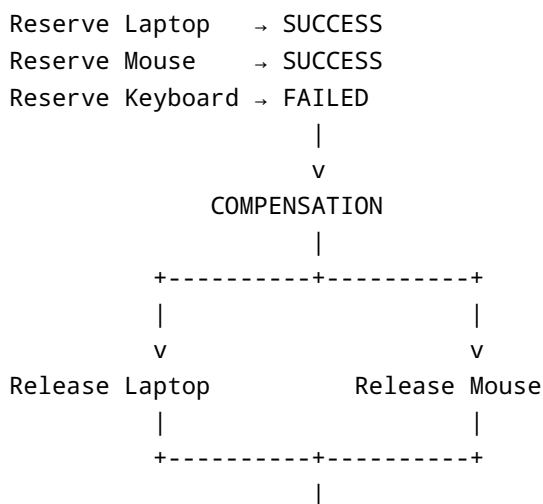
Order contains:

Line 1 → Laptop
Line 2 → Mouse
Line 3 → Keyboard

Suppose:

Line 1 → Reserved successfully
Line 2 → Reserved successfully
Line 3 → No stock

The Saga automatically compensates:



```
      v
Saga FAILED
  |
  v
Order FAILED
```

Every completed reservation has a corresponding release operation.

11. Idempotency

State-changing inter-service requests use an `Idempotency-Key`.

For example:

```
Idempotency-Key: SAGA-123:reserve:1
```

If the same request is sent again with the same key and payload, the previously stored result is returned.

This prevents:

```
Request 1 → Reserve 5 items
Request 2 → Reserve 5 items again
```

from accidentally reserving 10 items.

If the same idempotency key is reused with a different payload, the request is rejected as an idempotency conflict.

12. Retry and Circuit Breaker

Inter-service HTTP calls are protected by retries and a circuit breaker.

Retry

The client attempts up to three requests.

```
Attempt 1
  |
failure
```

```
|  
wait  
|  
Attempt 2  
|  
failure  
|  
wait longer  
|  
Attempt 3
```

The delay uses exponential backoff.

Circuit Breaker

The circuit has three states:

```
CLOSED  
|  
| repeated failures  
v  
OPEN  
|  
| cooldown  
v  
HALF_OPEN  
|  
+---- success ----> CLOSED  
|  
+---- failure ----> OPEN
```

When OPEN, calls fail immediately instead of continuously waiting for an unavailable service.

13. Persistence and Concurrency

Each service owns its own JSON files.

For example:

```
inventory_service/data/
```

is only used by the Inventory Service.

Services never directly read another service's JSON database.

Writes use an atomic process:

```
Create temporary file
  |
  v
Write complete JSON
  |
  v
Flush + fsync
  |
  v
Atomic replace
```

An in-process lock is also used to protect concurrent reads and writes.

14. Audit Events

The Audit Service provides a passive event receiver.

Services can send events such as:

```
ORDER_CREATED
STATUS_CHANGED
STOCK_RESERVED
STOCK_RELEASED
STOCK_COMMITTED
SAGA_STEP
SAGA_COMPLETED
SAGA_FAILED
```

Events are stored in chronological order.

A client can request:

```
GET /orders/{id}/history
```

to determine what happened to an order.

15. Testing

The project uses pytest.

Run:

```
pytest -q
```

Tests cover:

- Inventory reservation
- Idempotency
- Order creation
- Saga model behavior
- Circuit breaker behavior

The tests are designed to run without requiring all services to be running.

16. Docker

Docker Compose is included as an optional local-orchestration feature.

Build and start:

```
docker compose up --build
```

Stop:

```
docker compose down
```

Docker is only used for local convenience and is not required for the architecture.

17. Full Order Lifecycle Example

Step 1 — Create Warehouse

```
curl -X POST http://localhost:5000/warehouses  
-H "Content-Type: application/json"
```



```
-d "{\"id\":\"WH1\",\"name\":\"Islamabad Warehouse\",\"region\":\"Islamabad\",  
  \"address\":\"I-9 Islamabad\"}"
```

Step 2 — Add Stock

```
curl -X POST http://localhost:5000/warehouses/WH1/items  
-H "Content-Type: application/json"  
-d "{\"id\":\"ITEM1\",\"sku\":\"PHONE-01\",\"description\":\"Smart Phone\",  
  \"quantity_on_hand\":10,\"unit_cost\":500}"
```

Step 3 — Place Order

```
curl -X POST http://localhost:5000/orders  
-H "Content-Type: application/json"  
-d "{\"customer_name\":\"Ali\",\"customer_address\":\"Islamabad\",\"lines\":  
  [{\"sku\":\"PHONE-01\",\"quantity\":2}]}"
```

Step 4 — Check Order

```
curl http://localhost:5000/orders/ORDER_ID
```

Step 5 — Check History

```
curl http://localhost:5000/orders/ORDER_ID/history
```

Step 6 — Retry a Failed Saga

```
curl -X POST http://localhost:5000/sagas/SAGA_ID/retry
```

18. Forced Failure and Compensation

To demonstrate Saga compensation, create an order requesting more inventory than is available.

For example, if only 10 phones exist:

```
curl -X POST http://localhost:5000/orders  
-H "Content-Type: application/json"
```

```
-d "{\"customer_name\":\"Test Customer\",\"customer_address\":\"Islamabad\",  
  \"lines\": [{\"sku\":\"PHONE-01\",\"quantity\":100}]}"
```

The reservation fails because sufficient stock is unavailable.

The Fulfillment Service then:

1. Detects failure.
2. Changes Saga state to COMPENSATING.
3. Releases all previously successful reservations.
4. Changes Saga state to FAILED.
5. Changes Order status to FAILED.
6. Records the failure in the Audit Service.

19. Custom Exceptions

The system handles domain and distributed-system errors including:

- Duplicate IDs
- Invalid state transitions
- Insufficient stock
- Invalid SKU
- Cancelling an already shipped order
- Retrying a non-failed Saga
- Malformed requests
- Downstream timeout
- Downstream non-2xx response
- Idempotency conflicts
- Open circuit breaker

Errors are returned as structured JSON responses.

Example:

```
{  
  "error": "insufficient_stock",  
  "message": "Insufficient stock for PHONE-01"  
}
```

20. Git Workflow

Initialize repository:

```
git init
```

Add files:

```
git add .
```

Commit:

```
git commit -m "feat: implement distributed warehouse services"
```

Create additional commits using conventional commit messages:

```
feat: add inventory reservation
feat: implement fulfillment saga
feat: add audit service
feat: add api gateway
fix: handle reservation compensation
test: add idempotency tests
test: add circuit breaker tests
docs: update README
```

Push to GitHub:

```
git branch -M main
git remote add origin YOUR_GITHUB_REPOSITORY_URL
git push -u origin main
```

21. Submission Checklist

Before submitting the project, verify:

- [] Five processes are present.
- [] Each backend service has its own JSON database.
- [] Services communicate through HTTP.

- ☐ API Gateway is the client entry point.
 - ☐ Inventory reservation is idempotent.
 - ☐ Reservation release is idempotent.
 - ☐ Saga orchestration works.
 - ☐ Saga compensation works.
 - ☐ Failed Saga retry works.
 - ☐ Retry with backoff works.
 - ☐ Circuit breaker works.
 - ☐ Audit events are stored.
 - ☐ Order history works.
 - ☐ Health endpoint works.
 - ☐ Report endpoint works.
 - ☐ Atomic JSON writes are implemented.
 - ☐ Concurrent file access is protected.
 - ☐ Pytest passes.
 - ☐ `facility_report.txt` exists.
 - ☐ README contains API documentation.
 - ☐ README contains architecture explanation.
 - ☐ README contains lifecycle examples.
 - ☐ Git history contains meaningful commits.
 - ☐ Repository is public on GitHub.
-

22. Conclusion

This project demonstrates a distributed warehouse fulfillment system using independent Python microservices. It focuses on real distributed-system challenges such as partial failures, service-to-service communication, Saga-based transactions, compensation, idempotency, retries, circuit breakers, persistence, concurrency, and event auditing.

The architecture keeps each service independent while the API Gateway provides a unified interface for clients.